PART 2 OF 5 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 2: discrete models.
Solutions to every exercise



Rates with Bip

STOCHASTIC CALCULUS, STEP BY STEP · PART 2 OF 5 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 2: Discrete models. Solutions

Rates with Bip

Stochastic Calculus, Step by Step. Part 2: Discrete models. Solutions.

© 2026 Rates with Bip. This book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International licence (CC BY-NC-ND 4.0): you may copy and share it unchanged, for non-commercial purposes, crediting Rates with Bip. Licence: creativecommons.org/licenses/by-nc-nd/4.0. The companion notebooks are free under the MIT licence at github.com/RatesWithBip/curriculum.

Part 2's teaching tree: $S_0 = 100$, up 10% or down 10% every four months for three steps, real-world chance of going up 0.6, $r = 3\%$ (continuously compounded), $K = 105$; and the course's $\sigma = 20\%$ where a lesson says so. Made up for teaching; every random number is simulated with a fixed seed.

This volume holds the full solutions to the exercises and self-test questions in Stochastic Calculus, Step by Step, Part 2. Try each exercise before reading its solution.

Every number in the solutions is computed by a script that checks itself with assertions; the code solutions are shown in full.

This book is for education only. It is not financial, investment, legal or tax advice.

Contents

How to use the solutions 5

7. Information and Measure 6

8. Conditional Expectation 11

9. The One-Period Binomial Model 17

10. The Multi-Period Binomial Model 22

11. Martingales and Risk-Neutral Pricing 28

12. Change of Measure and American Options 33

How to use the solutions

This volume holds the full solutions to *Stochastic Calculus, Step by Step, Part 2: Discrete models*: every exercise worked out, and the answers to the self-test questions at the end of each lesson. It is meant to be read **after** you have tried each one.

- **By-hand exercises** (the first six of each lesson) are worked in full. Each solution starts with **the idea** (the one thought that unlocks it), then goes through numbered steps with the reason for each, and ends with a check or an interpretation. If you are stuck, read only the idea and try again before reading the steps.
- **Code exercises** (the last two of each lesson) show a complete Python solution and exactly what it prints. Every number in this volume is produced by a script that checks itself with assertions, using fixed random seeds, so your own run should match to the digits shown when you use the same seeds.
- **Self-test answers** are short: a sentence or two, and a number where there is one.

Part 2's teaching tree is the same throughout: $S_0 = 100$; each four-month step multiplies the price by $u = 1.1$ (heads) or $d = 0.9$ (tails); three steps; real-world chance of heads $p = 0.6$; cash grows by $R = e^{0.01}$ per step; strike $K = 105$. The risk-neutral chance of heads is $\tilde{p} = (R - d)/(u - d) \approx 0.5503$.

On a tree, almost every answer can be checked by writing the eight worlds in a column and adding up. If your answer differs, first check the usual suspects: a missing division by R (or one too many), the real probability p used where $\tilde{p}$ belongs, and a conditional average that was not renormalised by the probability of its block.

This volume is for education only and is not financial advice.

SOLUTIONS TO LESSON 7

Information and Measure

Solutions to Exercises 7.1 to 7.8 and the Lesson 7 self-test.

Exercise 7.1

The idea. After one toss the worlds split into two blocks, H$**$ and T$**$. The events of $\mathcal{F}_1$ are the unions of blocks.

Step 1. List them. Each of the two blocks is in or out, giving $2^2 = 4$ events:

- neither block: $\emptyset$;
- only the first: $\{\mathrm{HHH}, \mathrm{HHT}, \mathrm{HTH}, \mathrm{HTT}\}$, "the first toss is H";
- only the second: $\{\mathrm{THH}, \mathrm{THT}, \mathrm{TTH}, \mathrm{TTT}\}$, "the first toss is T";
- both: Ω.

Step 2. Rule 1. Ω is in the list.

Step 3. Rule 2 (complements). The complement of $\emptyset$ is Ω, and of "first H" is "first T", and the other way round. Each is in the list.

Step 4. Rule 3 (unions). Any union of events in the list is again in it: a union with $\emptyset$ changes nothing, a union with Ω is Ω, and "first H" $\cup$ "first T" $= \Omega$.

So $\mathcal{F}_1$ is a σ-algebra.

Exercise 7.2

The idea. An event made of blocks is fixed by deciding, block by block, whether it is in.

Step 1. At most 2^b. Every event in the σ-algebra is a union of some of the blocks, and there are 2^b ways to choose which blocks (two choices for each of b blocks).

Step 2. Exactly 2^b. Two different choices give different events: if block B is in one choice but not the other, the worlds of B are in the first event but not in the second (blocks don't overlap, so they can't sneak in through another block).

Step 3. The n-toss tree. After n tosses the blocks are the possible first-n sequences, 2^n of them. So $\mathcal{F}_n$ has $2^{(2^n)}$ events: 2, 4, 16, 256, 65,536, ... for $n = 0, 1, 2, 3, 4$.

Step 4. $\mathcal{F}_{10}$. $2^{10} = 1024$ blocks, so 2^{1024} events, a number with 309 digits. Information grows very fast with each toss.

Exercise 7.3

The idea. A is in $\mathcal{F}_2$ only if no time-2 block is split by it.

Step 1. Which worlds are in A. $S_3 > 105$ needs at least two heads (Lesson 7): $A = \{\mathrm{HHH}, \mathrm{HHT}, \mathrm{HTH}, \mathrm{THH}\}$.

Step 2. Find a split block. The block HT$*$ contains HTH ($S_3 = 108.9$, in A) and HTT ($S_3 = 89.1$, not in A). So A is not a union of time-2 blocks: $A \notin \mathcal{F}_2$.

Step 3. World by world. A world's outcome is settled at the first time its block lies wholly inside or wholly outside A.

WORLD	IN A?	KNOWN AT TIME
HHH, HHT	yes	2 (both continuations end above 105)
HTH, THH	yes	3
HTT, THT	no	3
TTH, TTT	no	2 (both continuations end below 105)

After one toss it is never settled: block H$**$ contains HTT, which ends below 105, and block T$**$ contains THH, which ends above.

Exercise 7.4

The idea. Group the worlds by the value of S_2.

Step 1. The blocks. S_2 is 121 after two heads, 99 after one head, 81 after none:

- $\{S_2 = 121\} = \{\mathrm{HHH}, \mathrm{HHT}\}$;
- $\{S_2 = 99\} = \{\mathrm{HTH}, \mathrm{HTT}, \mathrm{THH}, \mathrm{THT}\}$;
- $\{S_2 = 81\} = \{\mathrm{TTH}, \mathrm{TTT}\}$.

Three blocks, so $\sigma(S_2)$ has $2^3 = 8$ events.

Step 2. $\sigma(S_2) \subseteq \mathcal{F}_2$. Each of the three blocks is a union of time-2 blocks (the middle one is HT$* \cup$ TH$*$), so every union of them is too.

Step 3. Not equal. "The first toss is H" is $\{\mathrm{HH}*\} \cup \{\mathrm{HT}*\}$, in $\mathcal{F}_2$. It is not a union of $\sigma(S_2)$-blocks, because it contains HTH but not THH, which share the block $\{S_2 = 99\}$.

Interpretation. Watching only the price, you lose the **order** of the moves: at 99 you can't tell whether the share went up then down, or down then up. For the share's future that doesn't matter (Lesson 11); for a payoff that depends on the path, it can.

Exercise 7.5

The idea. Write each set as a union of disjoint pieces and apply rule 3.

$P(\emptyset) = 0$. Ω and $\emptyset$ are disjoint and $\Omega \cup \emptyset = \Omega$. By rule 3, $P(\Omega) = P(\Omega) + P(\emptyset)$, so $P(\emptyset) = 0$.

$P(A^c) = 1 - P(A)$. A and A^c are disjoint with union Ω, so $P(A) + P(A^c) = P(\Omega) = 1$ (rules 3 and 2).

$P(A \cup B)$. Split into three disjoint pieces: $A \cap B^c$ (only A), $A \cap B$ (both), $A^c \cap B$ (only B). Then

$$P(A \cup B) = P(A \cap B^c) + P(A \cap B) + P(A^c \cap B).$$

Also $P(A) = P(A \cap B^c) + P(A \cap B)$ and $P(B) = P(A^c \cap B) + P(A \cap B)$. Adding these two counts $P(A \cap B)$ twice, so subtract it once:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Numbers. $P(S_3 > 105) = 0.216 + 3 \times 0.144 = 0.648$. With "first H" (0.6) and "both" (HHH, HHT, HTH: 0.504):

$$P(S_3 > 105 \text{ or first H}) = 0.648 + 0.6 - 0.504 = 0.744.$$

Check. The union is HHH, HHT, HTH, HTT, THH: $0.216 + 0.144 + 0.144 + 0.096 + 0.144 = 0.744$.

Exercise 7.6

The idea. "Has a tail appeared by time n?" is answered by the first n tosses.

Step 1. $\{\tau \le n\} \in \mathcal{F}_n$. $\tau \le n$ means "there is a T among the first n tosses". Whether that is true depends only on the first n tosses, so every time-n block is wholly inside or wholly outside the event: it is a union of time-n blocks. (A random time with this property is a **stopping time**, Lesson 12.)

Step 2. The other event. $\{\tau = 1\} = \{\mathrm{T}{*}{*}\}$ and $\{\tau = 3\} = \{\mathrm{HHT}\}$. Their union contains HHT but not HHH, which is in the same time-2 block HH$*$. So it is not in $\mathcal{F}_2$: at time 2, after HH, you can't tell whether the first tail will come at step 3.

Step 3. Probabilities. The first tail is at step n when the first $n-1$ tosses are H and toss n is T:

n	EVENT	PROBABILITY
1	T	$0.4 = 0.400$
2	HT	$0.6 \times 0.4 = 0.240$
3	HHT	$0.6^2 \times 0.4 = 0.144$
4 (none)	HHH	$0.6^3 = 0.216$

Check. $0.4 + 0.24 + 0.144 + 0.216 = 1$.

Exercise 7.7

The time-2 σ-algebra of the four-toss tree, checked by brute force.

```
import itertools

worlds = ["".join(w) for w in itertools.product("HT", repeat=4)]   # 16 worlds of four tosses
Omega = frozenset(worlds)

blocks = {}
for w in worlds:                                    # time-2 blocks: same first two tosses
    blocks.setdefault(w[:2], []).append(w)
blocks = list(blocks.values())

F2 = set()
for pick in itertools.product([0, 1], repeat=len(blocks)):
    F2.add(frozenset(w for b, k in zip(blocks, pick) if k for w in b))

has_omega = Omega in F2
complements = all(Omega - A in F2 for A in F2)
unions = all(A | B in F2 for A in F2 for B in F2)
print(f"{len(blocks)} blocks, {len(F2)} events")
print("contains Omega:", has_omega, "| closed under complements:", complements, "| closed un-
der unions:", unions)
```

What it prints.

```
4 blocks, 16 events
contains Omega: True | closed under complements: True | closed under unions: True
```

Why 16. After two tosses there are 4 blocks (HH, HT, TH, TT), each containing $2^2 = 4$ worlds of the four-toss tree, so $2^4 = 16$ events. The number of events depends on the number of blocks, not on how many worlds each block contains: two more tosses in the future do not change what is known at time 2.

Exercise 7.8

Simulated frequencies against exact probabilities.

```
import numpy as np

p, u, d, K = 0.6, 1.1, 0.9, 105.0
rng = np.random.default_rng(8)
H = rng.random((200_000, 3)) < p                    # True = heads
first_two = np.char.add(np.where(H[:, 0], "H", "T"), np.where(H[:, 1], "H", "T"))
for block in ("HH", "HT", "TH", "TT"):
    exact = p ** block.count("H") * (1 - p) ** block.count("T")
    print(f"block {block}*: simulated {np.mean(first_two == block):.4f}, exact {exact:.4f}")
S3 = 100 * u ** H.sum(1) * d ** (3 - H.sum(1))
print(f"P(S3 > 105): simulated {np.mean(S3 > K):.4f}, exact {3 * p**2 * (1 - p) + p**3:.4f}")
```

What it prints.

```
block HH*: simulated 0.3614, exact 0.3600
block HT*: simulated 0.2400, exact 0.2400
block TH*: simulated 0.2383, exact 0.2400
block TT*: simulated 0.1603, exact 0.1600
P(S3 > 105): simulated 0.6469, exact 0.6480
```

Reading the output. Each block's frequency is within about 0.002 of its exact probability (0.6^2, 0.6×0.4, ...). With 200,000 runs the standard deviation of a frequency near 0.36 is $\sqrt{0.36 \times 0.64 / 200{,}000} \approx 0.001$ (Lesson 6), so these gaps are the size we should expect.

Self-test

1. 4: $\emptyset$, "first H", "first T" and Ω. Two blocks give 2^2 events.

2. No. The time-2 block HH$*$ contains HHH, with $S_3 = 133.1$, and HHT, with $S_3 = 108.9$: at time 2 you can't tell which.

3. Knowing the first two tosses. $\sigma(S_2) \subseteq \mathcal{F}_2$, and it is strictly smaller: at the price 99 you don't know whether the share went up then down or down then up.

SOLUTIONS TO LESSON 8

Conditional Expectation

Solutions to Exercises 8.1 to 8.8 and the Lesson 8 self-test.

Exercise 8.1

The idea. Inside a time-1 block, average S_2 over the worlds that are still possible, with their probabilities renormalised by the block's probability.

Block H ($P = 0.6$). S_2 depends only on the first two tosses: HH gives 121, HT gives 99. Within block H, the second toss is H with probability $0.36/0.6 = 0.6$ and T with probability $0.24/0.6 = 0.4$. So

$$E[S_2 \mid \mathcal{F}_1](\mathrm{H}) = 0.6 \times 121 + 0.4 \times 99 = 72.6 + 39.6 = 112.2.$$

Block T. TH gives 99 and TT gives 81:

$$E[S_2 \mid \mathcal{F}_1](\mathrm{T}) = 0.6 \times 99 + 0.4 \times 81 = 59.4 + 32.4 = 91.8.$$

Check. $1.02 \times 110 = 112.2$ and $1.02 \times 90 = 91.8$. ✓

Exercise 8.2

The idea. S_{n+1} is S_n times a factor decided by the next toss alone.

Step 1. Write $S_{n+1} = S_n G_{n+1}$, with $G_{n+1} = u$ after heads and d after tails.

Step 2. S_n is known at time n, so take it out (Rule 2): $E[S_{n+1} \mid \mathcal{F}_n] = S_n \, E[G_{n+1} \mid \mathcal{F}_n]$.

Step 3. Toss $n+1$ is independent of the first n tosses (Rule 3): $E[G_{n+1} \mid \mathcal{F}_n] = E[G_{n+1}] = pu + (1-p)d$.

So $E[S_{n+1} \mid \mathcal{F}_n] = \big(pu + (1-p)d\big) S_n = 1.02 \, S_n$.

Flat on average. The price is expected to stay put when $pu + (1-p)d = 1$, that is $p(u-d) = 1-d$:

$$p = \frac{1-d}{u-d} = \frac{0.1}{0.2} = 0.5.$$

A fair coin, which makes sense: the moves are symmetric, $\pm 10\%$.

Exercise 8.3

The idea. S_1 is known at time 1, so it comes out as a factor; then average with the tower rule.

Step 1. By Rule 2 and the lesson's result $E[S_3 \mid \mathcal{F}_1] = 1.02^2 S_1$:

$$E[S_1 S_3 \mid \mathcal{F}_1] = S_1 \, E[S_3 \mid \mathcal{F}_1] = 1.0404 \, S_1^2.$$

It is 12588.84 in block H ($S_1 = 110$) and 8427.24 in block T ($S_1 = 90$).

Step 2. By the tower rule, $E[S_1 S_3] = E\big[E[S_1 S_3 \mid \mathcal{F}_1]\big] = 1.0404 \, E[S_1^2]$, and

$$E[S_1^2] = 0.6 \times 110^2 + 0.4 \times 90^2 = 7260 + 3240 = 10500.$$

So $E[S_1 S_3] = 1.0404 \times 10{,}500 = 10924.2$.

Check. Equivalently, average the two block values: $0.6 \times 12{,}588.84 + 0.4 \times 8{,}427.24 = 10{,}924.2$.

Exercise 8.4

The idea. G is decided by the third toss alone.

Step 1. Independence. $G = S_3/S_2$ is u if the third toss is H and d if T. The third toss is independent of the first two, so G is independent of $\mathcal{F}_2$: knowing the first two tosses tells you nothing about it.

Step 2. By Rule 3, $E[G \mid \mathcal{F}_2] = E[G] = 0.6 \times 1.1 + 0.4 \times 0.9 = 1.02$, the same in all four blocks.

Step 3. $S_3 = S_2 G$, with S_2 known at time 2 (Rule 2), so

$$E[S_3 \mid \mathcal{F}_2] = S_2 \, E[G \mid \mathcal{F}_2] = S_2 \, E[G].$$

Both rules are needed: Rule 2 because S_2 is known, Rule 3 because G is independent.

Exercise 8.5

The idea. Average the time-2 estimates over the second toss, inside block H.

Step 1. The time-2 estimates. $E[S_3 \mid \mathcal{F}_2] = 1.02 \, S_2$: on HH ($S_2 = 121$) it is 123.42, on HT ($S_2 = 99$) it is 100.98.

Step 2. Average them in block H. Given the first toss was H, the second is H with probability 0.6:

$$0.6 \times 123.42 + 0.4 \times 100.98 = 74.052 + 40.392 = 114.444.$$

Step 3. Compare. $E[S_3 \mid \mathcal{F}_1](\mathrm{H}) = 1.0404 \times 110 = 114.444$. ✓ The tower rule: averaging tomorrow's estimate with today's information gives today's estimate.

Exercise 8.6

The idea. Both sides are sums over the four worlds of block H; the left uses the block's average, the right the individual payoffs.

Step 1. $E[X \mid \mathcal{F}_1]$**.** From the layer-by-layer tree of the lesson, it is 11.988 on H and 1.404 on T.

Step 2. Left side. $E[X \mid \mathcal{F}_1]\mathbf{1}_A$ is 11.988 on the four H-worlds and 0 elsewhere, and the H-worlds have total probability 0.6:

$$E\big[E[X \mid \mathcal{F}_1]\mathbf{1}_A\big] = 0.6 \times 11.988 = 7.1928.$$

Step 3. Right side. $X\mathbf{1}_A$ is the payoff in the H-worlds: 28.1 (HHH), 3.9 (HHT), 3.9 (HTH), 0 (HTT). Weighting by their probabilities:

$$\begin{aligned} E[X\mathbf{1}_A] &= 28.1(0.216) + 3.9(0.144) + 3.9(0.144) + 0 \\ &= 6.0696 + 0.5616 + 0.5616 = 7.1928. \end{aligned}$$

The two sides agree. ✓

Exercise 8.7

A general conditional expectation on the three-toss tree, and the tower rule for a random payoff.

```
import itertools

import numpy as np

p = 0.6
worlds = ["".join(w) for w in itertools.product("HT", repeat=3)]
prob = {w: p ** w.count("H") * (1 - p) ** w.count("T") for w in worlds}


def cond_exp(X, n):
    """E[X | F_n] for X a dict world -> value: average over the worlds that share the first n tosses."""
    res = {}
    for w in worlds:
        block = [v for v in worlds if v[:n] == w[:n]]
        res[w] = sum(X[v] * prob[v] for v in block) / sum(prob[v] for v in block)
    return res


rng = np.random.default_rng(87)
X = dict(zip(worlds, rng.normal(0, 10, len(worlds))))      # a random payoff, one value per world
lhs = cond_exp(cond_exp(X, 2), 1)
rhs = cond_exp(X, 1)
for w in ("HHH", "THH"):
    print(f"block {w[0]}: E[E[X|F2]|F1] = {lhs[w]:.6f}, E[X|F1] = {rhs[w]:.6f}")
print(f"largest gap over all worlds: {max(abs(lhs[w] - rhs[w]) for w in worlds):.1e}")
```

What it prints.

```
block H: E[E[X|F2]|F1] = 5.095197, E[X|F1] = 5.095197
block T: E[E[X|F2]|F1] = -1.809905, E[X|F1] = -1.809905
largest gap over all worlds: 8.9e-16
```

Why a random payoff. A rule checked on one nice payoff (like S_3, which has a pattern) could hold by coincidence. With eight unrelated random numbers there is no pattern to lean on, so agreement to rounding noise is good evidence that the code, and the rule, are right.

Exercise 8.8

The running maximum, conditioned on the path and on the price.

```
u, d = 1.1, 0.9
path = lambda w: [100 * u ** w[:n].count("H") * d ** w[:n].count("T") for n in range(4)]
M3 = {w: max(path(w)) for w in worlds}                          # the highest price seen, S0
included
S2 = {w: path(w)[2] for w in worlds}

on_F2 = cond_exp(M3, 2)
on_S2 = {}
for w in worlds:
    same_price = [v for v in worlds if abs(S2[v] - S2[w]) < 1e-9]
    on_S2[w] = sum(M3[v] * prob[v] for v in same_price) / sum(prob[v] for v in same_price)
for w in worlds:
    print(f"{w}: path {[round(x, 2) for x in path(w)]}, E[M3|F2] = {on_F2[w]:.4f}, E[M3|S2] =
{on_S2[w]:.4f}")
```

What it prints.

```
HHH: path [100.0, 110.0, 121.0, 133.1], E[M3|F2] = 128.2600, E[M3|S2] = 128.2600
HHT: path [100.0, 110.0, 121.0, 108.9], E[M3|F2] = 128.2600, E[M3|S2] = 128.2600
HTH: path [100.0, 110.0, 99.0, 108.9], E[M3|F2] = 110.0000, E[M3|S2] = 107.6700
HTT: path [100.0, 110.0, 99.0, 89.1], E[M3|F2] = 110.0000, E[M3|S2] = 107.6700
THH: path [100.0, 90.0, 99.0, 108.9], E[M3|F2] = 105.3400, E[M3|S2] = 107.6700
THT: path [100.0, 90.0, 99.0, 89.1], E[M3|F2] = 105.3400, E[M3|S2] = 107.6700
TTH: path [100.0, 90.0, 81.0, 89.1], E[M3|F2] = 100.0000, E[M3|S2] = 100.0000
TTT: path [100.0, 90.0, 81.0, 72.9], E[M3|F2] = 100.0000, E[M3|S2] = 100.0000
```

Where they differ. In the four worlds with $S_2 = 99$, that is, HTH, HTT, THH and THT.

- Given the path (the $\mathcal{F}_2$ block HT$*$), you know the share already reached 110, so $M_3 \geq 110$; the third step can only take the price to 108.9 or 89.1, below 110, so $E[M_3 \mid \mathcal{F}_2] = 110$.
- Given the path TH$*$, the highest so far is 100; after one more step it is 108.9 (heads) or still 100 (tails): $0.6 \times 108.9 + 0.4 \times 100 = 105.34$.
- Given only the price 99, you don't know which of the two paths you are on. Each has probability 0.24, so they count equally, and $E[M_3 \mid S_2 = 99] = \frac{1}{2}(110 + 105.34) = 107.67$.

Why. The maximum remembers the path; the price doesn't. A payoff on M_3 (a **lookback** option, Lesson 11) cannot be valued at a node of the recombining tree: it needs the path, or at least the pair (price, highest so far).

Self-test

1. A random variable. It is the average of S_3 over the time-1 block you will be in: 114.444 if the first toss is heads, 93.636 if tails.

2. $E[X]$, by the tower rule. For $X = S_3$, $E[S_3] = 106.1208$.

3. Y $E[X \mid \mathcal{F}_1]$: what is known comes out as a factor.

SOLUTIONS TO LESSON 9

The One-Period Binomial Model

Solutions to Exercises 9.1 to 9.8 and the Lesson 9 self-test.

Exercise 9.1

The idea. Same two equations as for the call, with the put's payoffs $V_u = 0$, $V_d = 15$.

Step 1. Δ.

$$\Delta = \frac{V_u - V_d}{S_0 u - S_0 d} = \frac{0 - 15}{110 - 90} = -0.75.$$

Step 2. B. From the heads equation, $\Delta \times 110 + BR = 0$, so $BR = 0.75 \times 110 = 82.5$ and

$$B = \frac{82.5}{R} = 81.6791.$$

Step 3. Price. $V_0 = \Delta S_0 + B = -75 + 81.6791 = 6.6791$.

Check both outcomes. After heads: $-0.75 \times 110 + 82.5 = 0$ ✓. After tails: $-0.75 \times 90 + 82.5 = 15$ ✓.

Why Δ is negative. The put pays more when the share falls. To copy something that moves opposite to the share, you need a short position in the share: here, three quarters of a share sold short, with the proceeds (and more) in cash.

Exercise 9.2

The idea. If one asset beats the other in every outcome, buy the winner with money borrowed from (or raised by selling) the loser.

$R = 1.12$**: cash beats the share in both outcomes.** Sell one share short (receive 100) and deposit the 100. Today: net zero. In four months the deposit is worth 112, and you buy back the share for 110 or 90:

OUTCOME	DEPOSIT	BUY BACK SHARE	PROFIT
Heads	112	110	2
Tails	112	90	22

Positive in both: an arbitrage.

$R = 0.88$**: the share beats cash in both outcomes.** Borrow 100 (repay 88) and buy one share:

OUTCOME	SELL SHARE	REPAY LOAN	PROFIT
Heads	110	88	22
Tails	90	88	2

(A gross return below 1 means a negative interest rate; the point is only that R must lie strictly between d and u.)

Exercise 9.3

The idea. Solve the two linear equations, then collect the coefficients of V_u and V_d.

Step 1. Subtract the equations $\Delta S_0 u + BR = V_u$ and $\Delta S_0 d + BR = V_d$:

$$\Delta S_0(u-d) = V_u - V_d \quad \Longrightarrow \quad \Delta = \frac{V_u - V_d}{S_0(u-d)}.$$

Step 2. Find B from the second equation (either works):

$$\begin{aligned} BR &= V_d - \Delta S_0 d = V_d - \frac{(V_u - V_d)\,d}{u-d} \\ &= \frac{V_d(u-d) - V_u d + V_d d}{u-d} = \frac{uV_d - dV_u}{u-d}. \end{aligned}$$

Step 3. The price. $V_0 = \Delta S_0 + B$:

$$\begin{aligned} V_0 &= \frac{V_u - V_d}{u-d} + \frac{uV_d - dV_u}{R(u-d)} = \frac{R(V_u - V_d) + uV_d - dV_u}{R(u-d)} \\ &= \frac{(R-d)V_u + (u-R)V_d}{R(u-d)}. \end{aligned}$$

Step 4. Read off the weights. With $\tilde{p} = (R-d)/(u-d)$ and $1-\tilde{p} = (u-R)/(u-d)$,

$$V_0 = \frac{\tilde{p}V_u + (1-\tilde{p})V_d}{R}.$$

Check with the call. $V_u = 5$, $V_d = 0$: $V_0 = \tilde{p} \times 5/R = 0.5503 \times 5/1.01005 = 2.7239$, as in the lesson.

Exercise 9.4

The idea. $\tilde{p}$ is a fraction whose numerator and denominator are distances on the number line.

Step 1. Between 0 and 1. Since $u > d$, the denominator $u - d$ is positive. So $\tilde{p} > 0 \iff R - d > 0 \iff R > d$, and $\tilde{p} < 1 \iff R - d < u - d \iff R < u$. Together: $0 < \tilde{p} < 1 \iff d < R < u$.

Step 2. The share's discounted average.

$$\tilde{p}u + (1 - \tilde{p})d = \tilde{p}(u - d) + d = (R - d) + d = R,$$

so $\big(\tilde{p}S_0 u + (1 - \tilde{p})S_0 d\big)/R = S_0 R/R = S_0$.

Numbers. $\tilde{p} = 0.5503$ and $(0.5503 \times 110 + (1 - 0.5503) \times 90)/R = 100.00$.

Interpretation. $d < R < u$ is exactly the condition for $\tilde{p}$ to be a probability. When it holds, the share's price is its discounted risk-neutral average: the share is "fairly priced" under $\tilde{p}$.

Exercise 9.5

The idea. A digital is just another pair of payoffs: $V_u = 10$, $V_d = 0$.

Δ. $(10 - 0)/(110 - 90) = 0.5$ shares.

B. $(10 - 0.5 \times 110)/R = -45/R =$ -44.5522: borrow.

Price. $0.5 \times 100 + B =$ 5.4478, or directly $\tilde{p} \times 10/R =$ 5.4478.

Interpretation. Paying 10 in the heads world costs $10\tilde{p}/R$: the discounted risk-neutral probability of heads, times 10. A digital's price *is* a (discounted) risk-neutral probability, which is why digitals are used to read the risk-neutral distribution off market prices.

Exercise 9.6

The idea. The quote is above the cost of the copy: sell the expensive thing, buy the cheap one.

Step 1. Today.

- Sell the call to the dealer: receive 3.10.
- Buy the copy: 0.25 shares for 25.00, paid with 2.7239 of the 3.10 plus a loan of 22.2761.
- Left in your pocket: $3.10 - 2.7239 = 0.3761$.

Step 2. In four months. The loan has grown to 22.50.

- After heads: the call costs you $110 - 105 = 5$. Your shares are worth $0.25 \times 110 = 27.5$; repay 22.5; left 5, which pays the call. Net 0.
- After tails: the call expires worthless. Your shares are worth $0.25 \times 90 = 22.5$; repay 22.5. Net 0.

Result. A profit of 0.3761 per call today, with no risk afterwards.

Exercise 9.7

A one-step pricer, and put-call parity across strikes.

```
import math


def one_period(S0, u, d, R, payoff):
    """Copy the payoff with shares and cash over one step; return Delta, cash B and the
price."""
    Vu, Vd = payoff(S0 * u), payoff(S0 * d)
    Delta = (Vu - Vd) / (S0 * u - S0 * d)
    B = (Vu - Delta * S0 * u) / R
    return Delta, B, Delta * S0 + B


S0, u, d, R = 100.0, 1.1, 0.9, math.exp(0.03 / 3)
for K in range(92, 109, 4):
    C = one_period(S0, u, d, R, lambda s: max(s - K, 0))[2]
    P = one_period(S0, u, d, R, lambda s: max(K - s, 0))[2]
    print(f"K = {K}: C = {C:7.4f}, P = {P:7.4f}, C - P = {C - P:8.4f}, S0 - K/R = {S0 - K /
R:8.4f}")
```

What it prints.

```
K = 92: C =  9.8060, P =  0.8905, C - P =   8.9154, S0 - K/R =   8.9154
K = 96: C =  7.6269, P =  2.6716, C - P =   4.9552, S0 - K/R =   4.9552
K = 100: C =  5.4478, P =  4.4527, C - P =   0.9950, S0 - K/R =   0.9950
K = 104: C =  3.2687, P =  6.2338, C - P =  -2.9652, S0 - K/R =  -2.9652
K = 108: C =  1.0896, P =  8.0149, C - P =  -6.9254, S0 - K/R =  -6.9254
```

Why parity holds for every strike. For any K, a call minus a put pays $S_1 - K$ in both outcomes; so does one share minus a loan repaying K. The function copies each option separately, and the two copies combine to exactly that portfolio.

Exercise 9.8

Where the naive value meets the price.

```
import numpy as np

Vu, Vd = 5.0, 0.0
price = one_period(S0, u, d, R, lambda s: max(s - 105, 0))[2]
ps = np.linspace(0, 1, 100_001)
naive = (ps * Vu + (1 - ps) * Vd) / R
cross = ps[np.argmin(np.abs(naive - price))]
print(f"no-arbitrage price {price:.4f}; the naive value equals it at p = {cross:.4f}")
print(f"risk-neutral probability (R - d)/(u - d) = {(R - d) / (u - d):.4f}")
```

What it prints.

```
no-arbitrage price 2.7239; the naive value equals it at p = 0.5503
risk-neutral probability (R - d)/(u - d) = 0.5503
```

Reading it. The naive value pV_u/R is a straight line in p, from 0 at $p = 0$ to $5/R$ at $p = 1$; the price is flat. They cross at $p = 0.5503$, which is $\tilde{p}$. So the risk-neutral probability is the one belief about heads under which the naive "average payoff, discounted" happens to be right.

Self-test

1. No. The price, 2.7239, is the cost of the copy, which uses only u, d, R and the payoffs. The real probability doesn't enter.

2. The copy holds a quarter of a share. Equivalently: if the share moves by 1 (in the range of this step), the call's value moves by about 0.25.

3. $d < R < u$. Otherwise either cash or the share wins in every outcome, and borrowing one to buy the other is an arbitrage.

SOLUTIONS TO LESSON 10

The Multi-Period Binomial Model

Solutions to Exercises 10.1 to 10.8 and the Lesson 10 self-test.

Exercise 10.1

The idea. Start at the payoffs after two steps and apply the one-step formula $V = (\tilde{p}V_u + (1-\tilde{p})V_d)/R$ twice, with $\tilde{p} = 0.5503$.

Step 1. Expiry (eight months). The prices are 121, 99 and 81, so the call pays 16, 0 and 0.

Step 2. Four months.

- Node 110 (next: 16 or 0): $V = (0.5503 \times 16)/R = 8.7164$.
- Node 90 (next: 0 or 0): $V = 0$.

Step 3. Today.

$$V_0 = \frac{0.5503 \times 8.7164 + (1 - 0.5503) \times 0}{R} = 4.7485.$$

Check. The two-step call is worth less than the three-step one (6.0893): it has less time to finish above 105.

Exercise 10.2

The idea. At each node, $\Delta = (V_u - V_d)/(S_u - S_d)$ and $B = V - \Delta S$.

NODE	Δ	B
Today, 100	$(8.7164 - 0)/(110 - 90) = 0.4358$	-38.8336
Four months, 110	$(16 - 0)/(121 - 99) = 0.7273$	-71.2836
Four months, 90	$(0 - 0)/(99 - 81) = 0$	0

Why $\Delta = 0$ at the lower node. From 90, one more step reaches 99 or 81, both below the strike. The call is certain to expire worthless, so it is worth 0 and its copy is empty: no shares, no cash.

Check at node 110. After heads: $0.7273 \times 121 + B_H R = 88.0 - 72.0 = 16$ ✓; after tails: $0.7273 \times 99 - 72.0 = 0$ ✓ (with $B_H R = -72$ exactly, since $B_H = -72/R$).

Exercise 10.3

The idea. Same backward induction with the put's payoff $\max(105 - S_3, 0)$.

Step 1. Expiry. At 133.1, 108.9, 89.1, 72.9 the put pays 0, 0, 15.9, 32.1.

Step 2. Eight months.

- Node 121: both next values are 0, so 0.
- Node 99: $\big((1 - 0.5503) \times 15.9\big)/R = 7.0799$.
- Node 81: $\big(0.5503 \times 15.9 + (1 - 0.5503) \times 32.1\big)/R = 22.9552$.

Step 3. Four months.

- Node 110: $\big((1 - 0.5503) \times 7.0799\big)/R = 3.1525$.
- Node 90: $\big(0.5503 \times 7.0799 + (1 - 0.5503) \times 22.9552\big)/R = 14.0783$.

Step 4. Today. $\big(0.5503 \times 3.1525 + (1 - 0.5503) \times 14.0783\big)/R = 7.9861$.

Parity. $C - P = 6.0893 - 7.9861 =$ -1.8968, and $S_0 - 105/R^3 = 100 - 101.8968 =$ -1.8968. ✓

Exercise 10.4

The idea. With $N = 2$ there are three end nodes, reached by 1, 2 and 1 paths.

$$V_0 = \frac{1}{R^2}\Big(\tilde{p}^{\,2} \times 16 + 2\tilde{p}(1 - \tilde{p}) \times 0 + (1 - \tilde{p})^2 \times 0\Big) = \frac{\tilde{p}^{\,2} \times 16}{R^2}.$$

Numbers. $\tilde{p}^{\,2} = 0.302776$ and $1/R^2 = 0.980199$, so $V_0 = 0.302776 \times 16 \times 0.980199 = 4.7485$, the answer to Exercise 10.1. ✓

Exercise 10.5

The idea. The old copy was built to be worth exactly the option's value after heads; the new copy costs exactly that value.

Step 1. The old portfolio after heads. Today's copy holds $\Delta_0 = 0.4537$ shares and cash $B_0 =$ -39.2815. After heads:

- shares: $0.4537 \times 110 = 49.9079$;
- cash: $B_0 \times R =$ -39.6763;

- total: 10.2316.

Step 2. The new portfolio at node 110. It holds $\Delta = 0.6782$ shares and cash $B =$ -64.3691:

- shares: $0.6782 \times 110 = 74.6007$;
- cash: -64.3691;
- total: 10.2316.

Step 3. Compare. Both equal $V_{1,1} = 10.2316$. Buying the extra $0.6782 - 0.4537 = 0.2245$ shares (cost 24.69) is paid for by borrowing exactly 24.69 more. No money comes in or goes out: the hedge is self-financing.

Exercise 10.6

The idea. A forward is a share minus a fixed debt; neither part needs adjusting.

Step 1. Value. At step n, hold one share and owe $105/R^{3-n}$ in cash (which grows to 105 at the end). That portfolio pays $S_3 - 105$ in every world, so the forward is worth

$$F_n = S_n - \frac{105}{R^{3-n}}.$$

Today: $100 - 105/R^3 = 100 - 101.8968 =$ -1.8968: negative, because the agreed price 105 is above the share's forward value $100R^3$.

Step 2. Δ. At any node,

$$\Delta = \frac{F^{\mathrm{H}}_{n+1} - F^{\mathrm{T}}_{n+1}}{S_n u - S_n d} = \frac{S_n u - S_n d}{S_n u - S_n d} = 1,$$

because the debt term is the same after heads and tails and cancels.

Why no rebalancing. Δ is 1 at every node, so the copy never changes: buy one share and borrow, then wait. A payoff that is a straight line in S_3 needs a fixed hedge; the call's kink at 105 is what makes its Δ move.

Exercise 10.7

A CRR call pricer, around N = 50.

```
import math

import numpy as np

S0, K, r, sigma, T = 100.0, 105.0, 0.03, 0.20, 1.0


def crr_call(N):
    dt = T / N
    u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; R = math.exp(r * dt)
    q = (R - d) / (u - d)
    s = S0 * u ** np.arange(N, -1, -1) * d ** np.arange(0, N + 1)    # expiry prices, highest
first
    v = np.maximum(s - K, 0.0)
    for _ in range(N):
        v = (q * v[:-1] + (1 - q) * v[1:]) / R                       # one step back
    return v[0]


Ncdf = lambda x: 0.5 * (1 + math.erf(x / math.sqrt(2)))
d1 = (math.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
bs = S0 * Ncdf(d1) - K * math.exp(-r * T) * Ncdf(d1 - sigma * math.sqrt(T))
print(f"Black-Scholes: {bs:.4f}")
for N in range(50, 61):
    print(f"N = {N}: {crr_call(N):.4f}, error {crr_call(N) - bs:+.4f}")
```

What it prints.

```
Black-Scholes: 7.1281
N = 50: 7.1234, error -0.0046
N = 51: 7.1592, error +0.0311
N = 52: 7.1197, error -0.0083
N = 53: 7.1592, error +0.0312
N = 54: 7.1162, error -0.0119
N = 55: 7.1591, error +0.0311
N = 56: 7.1128, error -0.0153
N = 57: 7.1588, error +0.0308
N = 58: 7.1096, error -0.0185
N = 59: 7.1584, error +0.0304
N = 60: 7.1065, error -0.0216
```

The pattern. For odd N the error is about +0.031 and barely moves; for even N it is negative and grows in size (from −0.005 to −0.022) over this range. The two families take turns: zoom out (as in the lesson's figure) and each family drifts in slow waves while both close in on the Black-Scholes value. The cause is where the strike falls between the final nodes: the kink in the payoff is sampled differently on the odd and even grids. Averaging two neighbouring N removes most of the zigzag.

Exercise 10.8

The hedge along all eight paths, and a lazy hedge.

```
import itertools

u, d, R, K = 1.1, 0.9, math.exp(0.03 / 3), 105.0
q = (R - d) / (u - d)
S = lambda n, k: 100 * u**k * d ** (n - k)
V = {(3, k): max(S(3, k) - K, 0) for k in range(4)}
for n in (2, 1, 0):
    for k in range(n + 1):
        V[(n, k)] = (q * V[(n + 1, k + 1)] + (1 - q) * V[(n + 1, k)]) / R
delta = lambda n, k: (V[(n + 1, k + 1)] - V[(n + 1, k)]) / (S(n + 1, k + 1) - S(n + 1, k))


def run(world, lazy=False):
    shares, k = delta(0, 0), 0
    cash = V[(0, 0)] - shares * 100
    for n, toss in enumerate(world, start=1):
        cash *= R
        k += toss == "H"
        if n < 3 and not lazy:
            new = delta(n, k)
            cash -= (new - shares) * S(n, k)          # rebalance out of the cash account
            shares = new
    return shares * S(3, k) + cash - max(S(3, k) - K, 0)


for w in ("".join(t) for t in itertools.product("HT", repeat=3)):
    print(f"{w}: hedge error {run(w):+.2e}, lazy hedge error {run(w, lazy=True):+.4f}")
```

What it prints.

```
HHH: hedge error +1.42e-14, lazy hedge error -8.1892
HHT: hedge error +1.42e-14, lazy hedge error +5.0310
HTH: hedge error -1.07e-14, lazy hedge error +5.0310
HTT: hedge error -1.07e-14, lazy hedge error -0.0524
THH: hedge error -3.55e-15, lazy hedge error +5.0310
THT: hedge error -3.55e-15, lazy hedge error -0.0524
TTH: hedge error -1.79e-15, lazy hedge error -0.0524
TTT: hedge error -1.79e-15, lazy hedge error -7.4025
```

Reading it. The rebalanced hedge ends within 10^{-13} of the payoff in every world: rounding noise. The lazy hedge, which keeps today's 0.4537 shares to the end, misses by up to 8.19, larger than the option's price. Its errors also depend on the path: in HHH the call pays far more than the lazy hedge holds, and in TTT the lazy hedge loses on its shares while the call pays nothing. Hedging is not a one-off purchase; it is a strategy that follows the share.

Self-test

1. At expiry the option's value is known (its payoff). One step earlier, each node is a one-step problem whose two outcomes are now known. Working forward, the outcomes would still be unknown.

2. No. Each Δ uses only the option's values at the next two nodes and the share's prices there; the real probability appears nowhere.

3. $C - P = S_0 - K/R^N$ (with continuous compounding, $S_0 - Ke^{-rT}$). On the teaching tree: -1.8968.

SOLUTIONS TO LESSON 11

Martingales and Risk-Neutral Pricing

Solutions to Exercises 11.1 to 11.8 and the Lesson 11 self-test.

Exercise 11.1

The idea. On the tree, $E[S_{n+1} \mid \mathcal{F}_n] = g\,S_n$ with g the average one-step growth factor; compare g with 1.

Under $p = 0.6$. $g = 0.6 \times 1.1 + 0.4 \times 0.9 = 1.02 > 1$, so $E[S_{n+1} \mid \mathcal{F}_n] = 1.02\,S_n \geq S_n$ (prices are positive): a **submartingale**.

A martingale needs $g = 1$: $p \times 1.1 + (1-p) \times 0.9 = 1$ gives $0.2p = 0.1$, so $p = 0.5$.

Under $\tilde{p}$. $g = \tilde{p}u + (1-\tilde{p})d = R = 1.0101 > 1$. So S_n itself is a submartingale under $\tilde{p}$ too: it grows like cash. Only after dividing by R^n does it become a martingale.

Exercise 11.2

The idea. Split S_{n+1} into a known part and an independent part.

Step 1. $S_{n+1} = S_n G_{n+1}$, with $G_{n+1} \in \{u, d\}$ decided by toss $n+1$.

Step 2. Take out what is known. S_n is $\mathcal{F}_n$-measurable, so $\tilde{E}[S_{n+1} \mid \mathcal{F}_n] = S_n\,\tilde{E}[G_{n+1} \mid \mathcal{F}_n]$.

Step 3. Independence. Toss $n+1$ is independent of $\mathcal{F}_n$ under $\tilde{P}$ as well (the risk-neutral probability still treats the tosses as independent, each H with probability $\tilde{p}$). So $\tilde{E}[G_{n+1} \mid \mathcal{F}_n] = \tilde{p}u + (1-\tilde{p})d$.

Step 4. Use the definition of $\tilde{p}$. $\tilde{p}u + (1-\tilde{p})d = \tilde{p}(u-d) + d = (R-d) + d = R$. This uses only $\tilde{p} = (R-d)/(u-d)$, which is a probability because $d < R < u$.

Step 5. Discount.

$$\tilde{E}\left[\frac{S_{n+1}}{R^{n+1}} \,\middle|\, \mathcal{F}_n\right] = \frac{S_n R}{R^{n+1}} = \frac{S_n}{R^n}.$$

Exercise 11.3

The idea. One step of backward induction *is* one step of the martingale property.

Step 1. At a node at step n, backward induction sets $V_n = \big(\tilde{p}V_{n+1}^{\mathrm{H}} + (1-\tilde{p})V_{n+1}^{\mathrm{T}}\big)/R$. Given $\mathcal{F}_n$ (you are at that node), V_{n+1} is V_{n+1}^{H} with probability $\tilde{p}$ and V_{n+1}^{T} with probability $1-\tilde{p}$, so the bracket is $\tilde{E}[V_{n+1} \mid \mathcal{F}_n]$. Hence

$$V_n = \frac{\tilde{E}[V_{n+1} \mid \mathcal{F}_n]}{R} \quad \Longrightarrow \quad \frac{V_n}{R^n} = \tilde{E}\Big[\frac{V_{n+1}}{R^{n+1}} \,\Big|\, \mathcal{F}_n\Big].$$

Step 2. Chain the steps with the tower rule.

$$\begin{aligned}\frac{V_0}{R^0} &= \tilde{E}\Big[\frac{V_1}{R}\Big] = \tilde{E}\Big[\tilde{E}\Big[\frac{V_2}{R^2} \,\Big|\, \mathcal{F}_1\Big]\Big] \\ &= \tilde{E}\Big[\frac{V_2}{R^2}\Big] = \tilde{E}\Big[\frac{V_3}{R^3}\Big].\end{aligned}$$

So $V_0 = \tilde{E}[V_3]/R^3$, with V_3 the payoff.

Exercise 11.4

The idea. The tower rule turns a forecast several steps ahead into a chain of one-step forecasts.

Constant expectation. Take the expectation of $E[M_{n+1} \mid \mathcal{F}_n] = M_n$. By the tower rule the left side becomes $E[M_{n+1}]$, so $E[M_{n+1}] = E[M_n]$ for every n, and therefore $E[M_n] = E[M_0] = M_0$ (M_0 is known at time 0, a number).

Many steps. By induction on m. For $m = n+1$ it is the definition. Suppose $E[M_m \mid \mathcal{F}_n] = M_n$. Then, since $\mathcal{F}_n \subseteq \mathcal{F}_m$,

$$E[M_{m+1} \mid \mathcal{F}_n] = E\big[E[M_{m+1} \mid \mathcal{F}_m] \mid \mathcal{F}_n\big] = E[M_m \mid \mathcal{F}_n] = M_n.$$

Exercise 11.5

The idea. The lookback's payoff depends on the path, so work world by world: find the highest price on each path, its payoff, and weight by $\tilde{P}(\omega)$.

The lookback call, world by world.

WORLD	Highest M_3	PAYOFF	$\tilde{P}$	PAYOFF × $\tilde{P}$
HHH	133.1	28.1	0.1666	4.6815
HHT	121.0	16.0	0.1362	2.1788
HTH	110.0	5.0	0.1362	0.6809
HTT	110.0	5.0	0.1113	0.5565
THH	108.9	3.9	0.1362	0.5311
THT	100.0	0.0	0.1113	0.0000
TTH	100.0	0.0	0.1113	0.0000
TTT	100.0	0.0	0.0910	0.0000

Step 1. The highest prices. HHH climbs to 133.1; HHT reaches 121 before falling; HTH and HTT peak at 110 after the first step; THH ends at its peak, 108.9; the other three never beat the starting 100.

Step 2. Add the last column. The sum is 8.6288.

Step 3. Discount. $1/R^3 = 0.970446$, so the price is 8.3737.

Compare. The ordinary call is worth 6.0893. The lookback is worth more because it pays on the best price seen, not the last: in HHT, HTH, HTT and THH it pays at least as much as the call, and in HHT, HTT it pays where the call pays little or nothing.

Exercise 11.6

The idea. A self-financing portfolio is shares plus cash, and under $\tilde{p}$ both grow on average like cash.

Step 1. One step. Holding Δ_n shares (chosen at time n) and $X_n - \Delta_n S_n$ in cash,

$$X_{n+1} = \Delta_n S_{n+1} + (X_n - \Delta_n S_n)R.$$

Step 2. Forecast. Δ_n and X_n are known at time n; $\tilde{E}[S_{n+1} \mid \mathcal{F}_n] = RS_n$ (Exercise 11.2). So

$$\tilde{E}[X_{n+1} \mid \mathcal{F}_n] = \Delta_n RS_n + (X_n - \Delta_n S_n)R = RX_n,$$

and dividing by R^{n+1}, X_n/R^n is a martingale.

Step 3. No arbitrage. Suppose $X_0 = 0$ and $X_N \geq 0$ in every world. By constant expectation, $\tilde{E}[X_N/R^N] = X_0 = 0$. So $\sum_\omega \tilde{P}(\omega)X_N(\omega) = 0$, a sum of terms that are all ≥ 0. Each term must be 0, and since $\tilde{P}(\omega) > 0$ for every world, $X_N(\omega) = 0$ everywhere. The

portfolio never makes money: no arbitrage.

Where positivity mattered. If $\tilde{P}$ gave some world probability 0, the argument would say nothing about that world, and a strategy could profit there. That is why the risk-neutral probability must give every world positive weight (Lesson 12: equivalence).

Exercise 11.7

An Asian call by brute force over all paths.

```
import itertools
import math

S0, K, r, sigma, T = 100.0, 105.0, 0.03, 0.20, 1.0


def asian_brute(N):
    dt = T / N
    u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; R = math.exp(r * dt)
    q = (R - d) / (u - d)
    total = 0.0
    for path in itertools.product((True, False), repeat=N):     # all 2^N paths
        s, running, heads = S0, 0.0, 0
        for up in path:
            s *= u if up else d
            running += s
            heads += up
        total += q**heads * (1 - q) ** (N - heads) * max(running / N - K, 0.0)
    return total / R**N


for N in (4, 8, 12, 16):
    print(f"N = {N:2d}: {2**N:6d} paths, Asian call {asian_brute(N):.4f}")
print(f"N = 252 would need 2^252 = {float(2**252):.2e} paths")
```

What it prints.

```
N =  4:     16 paths, Asian call 4.2408
N =  8:    256 paths, Asian call 3.7018
N = 12:   4096 paths, Asian call 3.5117
N = 16:  65536 paths, Asian call 3.4203
N = 252 would need 2^252 = 7.24e+75 paths
```

Reading it. The number of paths doubles with each step: 2^N. Sixteen steps already need 65,536 paths; daily steps for a year would need about 7×10^{75}, more than the number of atoms in the Earth. The price also moves with N (averaging over more dates smooths the average, which lowers the option's value). In practice an Asian option is priced on an enlarged Markov state (price and running total) or by Monte Carlo.

Exercise 11.8

The martingale property at every node of a 50-step tree.

```
import numpy as np

N = 50
dt = T / N
u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; R = math.exp(r * dt)
q = (R - d) / (u - d)
values = [None] * (N + 1)
s = S0 * u ** np.arange(N, -1, -1) * d ** np.arange(0, N + 1)
values[N] = np.maximum(s - K, 0.0)
for n in range(N - 1, -1, -1):
    values[n] = (q * values[n + 1][:-1] + (1 - q) * values[n + 1][1:]) / R
gap = 0.0
for n in range(N):
    today = values[n] / R**n                                    # discounted value at each
node of step n
    forecast = (q * values[n + 1][:-1] + (1 - q) * values[n + 1][1:]) / R ** (n + 1)
    gap = max(gap, float(np.max(np.abs(forecast - today))))
print(f"price {values[0][0]:.4f}; largest martingale gap over {N * (N + 1) // 2} nodes:
{gap:.1e}")
```

What it prints.

```
price 7.1234; largest martingale gap over 1275 nodes: 2.8e-14
```

Reading it. The gap is rounding noise at all 1,275 nodes before expiry: on any CRR tree, the discounted option value is a martingale under the risk-neutral probability, for exactly the reason of Exercise 11.3. (The price, 7.1234, is the 50-step value of Lesson 10's convergence study.)

Self-test

1. A submartingale. Under $p = 0.6$ the discounted share grows on average by $1.02/R = 1.0099$ per step: the share is expected to beat cash, which is the reward for its risk.

2. Cash itself grows by R every step. Without discounting, every asset would drift up at least like cash, and nothing would be a martingale. Discounting measures prices in units of cash, removing the risk-free growth that every asset shares.

3. No. Its payoff depends on the average of the path; paths that meet at a node (HTH and THH both reach 108.9) can have different averages (105.97 and 99.30), so the node cannot hold a single value. Use all paths, a larger Markov state, or simulation.

SOLUTIONS TO LESSON 12

Change of Measure and American Options

Solutions to Exercises 12.1 to 12.8 and the Lesson 12 self-test.

Exercise 12.1

The idea. Both probabilities are products over the tosses, so their ratio is a product of one factor per toss.

Step 1. The two factors.

$$\frac{\tilde{p}}{p} = \frac{0.550251}{0.6} = 0.9171, \qquad \frac{1-\tilde{p}}{1-p} = \frac{1-0.550251}{0.4} = 1.1244.$$

Step 2. Z by number of heads.

HEADS	WORLDS	Z
3	HHH	$0.9171^3 = 0.7713$
2	HHT, HTH, THH	$0.9171^2 \times 1.1244 = 0.9456$
1	HTT, THT, TTH	$0.9171 \times 1.1244^2 = 1.1594$
0	TTT	$1.1244^3 = 1.4214$

Step 3. $E[Z]$. Weight each row by its real probability $(0.216,\ 3\times 0.144,\ 3\times 0.096,\ 0.064)$:

$$E[Z] = 0.1666 + 0.4085 + 0.3339 + 0.0910 = 1.0000.$$

Why exactly 1. Each row's term is $P(\text{row}) \times \tilde{P}/P = \tilde{P}(\text{row})$, and the risk-neutral probabilities add to 1.

Exercise 12.2

The idea. Write the $\tilde{P}$-average as a sum over worlds and insert $\tilde{P} = ZP$.

Proof. $\tilde{E}[X] = \sum_\omega X(\omega)\tilde{P}(\omega) = \sum_\omega X(\omega)\,Z(\omega)P(\omega) = E[ZX]$.

Check with $X = S_3$.

- $\tilde{E}[S_3] = 103.0455$ (the share grows like cash: $100R^3 = 103.0455$);
- $E[ZS_3] = 103.0455$. ✓

Exercise 12.3

The idea. Split Z into one factor per toss: the first n are known at time n, the rest are independent and average to 1.

Step 1. $Z = f_1 f_2 f_3$, with $f_i = \tilde{p}/p$ if toss i is H and $(1-\tilde{p})/(1-p)$ if T.

Step 2. Each factor averages to 1 under P:

$$E[f_i] = p\,\frac{\tilde{p}}{p} + (1-p)\,\frac{1-\tilde{p}}{1-p} = \tilde{p} + 1 - \tilde{p} = 1.$$

Numerically: $0.6 \times 0.9171 + 0.4 \times 1.1244 = 1.0000$.

Step 3. Condition on $\mathcal{F}_n$. $f_1, \ldots, f_n$ are known (take them out); $f_{n+1}, \ldots, f_3$ depend only on later tosses, independent of $\mathcal{F}_n$, and the expectation of a product of independent factors is the product of their expectations:

$$Z_n = E[Z \mid \mathcal{F}_n] = f_1 \cdots f_n \, E[f_{n+1}] \cdots E[f_3] = f_1 \cdots f_n.$$

That is $(\tilde{p}/p)^{\#\mathrm{H}}((1-\tilde{p})/(1-p))^{\#\mathrm{T}}$ over the first n tosses.

Step 4. Martingale. $Z_{n+1} = Z_n f_{n+1}$, so $E[Z_{n+1} \mid \mathcal{F}_n] = Z_n E[f_{n+1}] = Z_n$.

Exercise 12.4

The idea. A price is a weighted sum over worlds; a world with zero weight is a world whose payoffs are free.

Why equivalence is needed.

- If $\tilde{P}(A) = 0$ for an event with $P(A) > 0$: a security paying 1 on A would be priced $\tilde{P}(A)/R^N = 0$. Get it for free; it pays 1 with real probability $P(A) > 0$, and never loses. Arbitrage.
- If $P(A) = 0$ but $\tilde{P}(A) > 0$: the market would charge for payoffs in a world that cannot happen. Selling that security brings in money today with no real risk of paying out. Again an arbitrage.

So pricing probabilities must agree with reality about what is possible, while being free to disagree about how likely it is.

When $p = 1$. Then tails is impossible: the share goes up 10% every step for certain. But $\tilde{p} = 0.5503 < 1$ gives tails positive weight, so $\tilde{P}$ is not equivalent to P, and $Z = \tilde{P}/P$ is undefined on the tails worlds (division by 0). In fact the market itself has an arbitrage: borrow 100 and buy the share; after a step you have 110 for certain and owe only $100R = 101.005$. A share that *surely* beats cash is a free lunch, and no equivalent risk-neutral probability exists, as the first fundamental theorem says.

Exercise 12.5

The idea. The continuation value of the call is at least the value of a forward with the same strike, and the forward is worth more than exercising.

Step 1. Call ≥ forward. At expiry, $\max(S_3 - 105, 0) \geq S_3 - 105$ in every world. Prices preserve "pays at least as much in every world" (the pricing formula is an average with positive weights), so at step n the European call is worth at least the forward, $S_n - 105/R^{3-n}$ (Lesson 10, Exercise 10.6).

Step 2. Forward > exercise. $R > 1$, so $105/R^{3-n} < 105$ for $n < 3$, and

$$S_n - \frac{105}{R^{3-n}} > S_n - 105.$$

Paying 105 later is cheaper than paying it now: 103.9552 one step early, 102.9209 two steps, 101.8968 three.

Step 3. Never exercise early. Work backward. At step 2 the continuation value (the European one-step value) exceeds the intrinsic value whenever the latter is positive (Steps 1 and 2), and it is ≥ 0 otherwise; so the American value equals the European value there. The same argument then applies at step 1 and step 0. The American call is never exercised before expiry, and its price is the European call's, 6.0893.

Exercise 12.6

The idea. Backward induction with $\max(\text{intrinsic}, \text{continuation})$ at each node, over two steps.

Step 1. Expiry (eight months). At 121, 99 and 81 the put pays 0, 6 and 24.

Step 2. Four months.

- Node 110: continuation $\big((1-\tilde{p}) \times 6\big)/R = 2.6716$; intrinsic 0. **Wait**: 2.6716.
- Node 90: continuation $\big(\tilde{p} \times 6 + (1-\tilde{p}) \times 24\big)/R = 13.9552$; intrinsic 15. **Exercise**: 15.

Step 3. Today. Continuation $\big(\tilde{p} \times 2.6716 + (1-\tilde{p}) \times 15\big)/R = 8.1346$; intrinsic 5. Wait. **Price: 8.1346.**

Early exercise happens at the node 90, after a first tail. For comparison, the European two-step put is worth 7.6694.

Exercise 12.7

American and European puts, and the exercise boundary at six months.

```
import math

import numpy as np

S0, K, r, sigma, T = 100.0, 105.0, 0.03, 0.20, 1.0


def crr_put(N, american=True, boundary_at=None):
    dt = T / N
    u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; R = math.exp(r * dt)
    q = (R - d) / (u - d)
    s = S0 * u ** np.arange(N, -1, -1) * d ** np.arange(0, N + 1)
    v = np.maximum(K - s, 0.0)
    boundary = None
    for n in range(N - 1, -1, -1):
        s = s[:-1] / u
        v = (q * v[:-1] + (1 - q) * v[1:]) / R
        if american:
            exercise = K - s > v                                    # exercising beats holding
            v = np.where(exercise, K - s, v)
            if n == boundary_at and exercise.any():
                boundary = s[exercise].max()                        # highest price where ex-
ercising is best
    return v[0], boundary


for N in range(100, 1001, 100):
    am, b = crr_put(N, boundary_at=N // 2)
    eu = crr_put(N, american=False)[0]
    print(f"N = {N:4d}: American {am:.4f}, European {eu:.4f}, premium {am - eu:.4f}, "
          f"boundary at six months {b:.2f}")
```

What it prints.

```
N =  100: American 9.4875, European 9.0308, premium 0.4567, boundary at six months 85.21
N =  200: American 9.4825, European 9.0301, premium 0.4524, boundary at six months 84.39
N =  300: American 9.4780, European 9.0230, premium 0.4550, boundary at six months 85.07
N =  400: American 9.4817, European 9.0293, premium 0.4523, boundary at six months 85.21
N =  500: American 9.4795, European 9.0269, premium 0.4525, boundary at six months 85.13
N =  600: American 9.4759, European 9.0216, premium 0.4542, boundary at six months 84.93
N =  700: American 9.4788, European 9.0258, premium 0.4530, boundary at six months 84.68
N =  800: American 9.4795, European 9.0271, premium 0.4524, boundary at six months 84.39
N =  900: American 9.4789, European 9.0265, premium 0.4524, boundary at six months 85.21
N = 1000: American 9.4776, European 9.0247, premium 0.4529, boundary at six months 84.84
```

Reading it. The American put settles near 9.48 and the premium near 0.45. The **exercise boundary** at six months is about 85: below it, exercising at once beats waiting. It jumps around by a few tenths because only tree nodes can be tested, and their spacing changes with N; a finer answer needs interpolation between nodes, or the finite-difference grids of Lesson 29.

Exercise 12.8

Two exercise rules, simulated under the risk-neutral probability.

```
u, d, R, K = 1.1, 0.9, math.exp(0.03 / 3), 105.0
q = (R - d) / (u - d)
rng = np.random.default_rng(12)
heads = rng.random((200_000, 3)) < q                 # paths under the risk-neutral proba-
bility
S = 100 * np.cumprod(np.where(heads, u, d), axis=1)   # S1, S2, S3 on each path
payoff = lambda s: np.maximum(K - s, 0.0)

# best rule (from the tree): exercise at step 1 if the first toss is tails, otherwise hold to
the end
best = np.where(~heads[:, 0], payoff(S[:, 0]) / R, payoff(S[:, 2]) / R**3)

# exercise as soon as in the money
itm = payoff(S) > 0
first = np.where(itm.any(axis=1), itm.argmax(axis=1), 2)   # first step in the money, else
the last step
greedy = payoff(S[np.arange(len(S)), first]) / R ** (first + 1)

print(f"best rule:               {best.mean():.4f} (exact {8.3965:.4f})")
print(f"as soon as in the money: {greedy.mean():.4f} (exact {8.1346:.4f})")
```

What it prints.

```
best rule:               8.3917 (exact 8.3965)
as soon as in the money: 8.1383 (exact 8.1346)
```

Reading it. Both estimates are within simulation noise of the exact values (the standard error with 200,000 paths is about 0.015). The best rule, from the tree, earns the American price; exercising greedily at 99 after HT throws away about 0.26 of value. The simulation also shows why the rule must be a stopping time: each decision uses only the tosses seen so far.

Self-test

1. $E[Z] = 1$, because $E[Z] = \sum_\omega (\tilde{P}(\omega)/P(\omega))P(\omega) = \sum_\omega \tilde{P}(\omega) = 1$.

2. At the node 90 after a first tail (and at 81, which is reached only through 90, where the put has already been exercised). The early-exercise premium is 0.4104.

3. A supermartingale: at each node the value is at least the discounted continuation value, with strict inequality where exercise is optimal.